Image processing	Task No. 1
Task variant N5	
Day and time Monday 13:00 – 14:30 **Academic year 2026/2027**	**Full name: Aleksander Szaforski** **Full name: Kiyan Fatnassi** The text in blue contains instructions about what should be put into appropriate sections of the report. The instructions must be followed and the blue text itself should be deleted afterwards. The layout of this template must be respected; otherwise the report will be rejected. In this section the following information should be given: the number of the task, its variant, day and time of the classes, academic year and the full names of the group members.

Technical description of the application

Description:

The application is command-line project implemented in python that performs:

Basic image processing

Geometric transformation

Noise removal

Standard python libraries:

sys for processing command-line arguments

math for calculations (logarithmic in noise metrics)

External library used is PIL for image loading and saving.

Memory: After loading image it is converted into grayscale (one value per pixel) or RGB (3 values per pixel), latter the pixels are stored in one-dimensional list.

In this section the technical description of the application should be placed. In the case of using an external library, specify how it is used, what structures are used for storing the image in memory, etc.

Description of implementation of basic image operations

- Description (B1):

  Command input: python myWonderfulImProcApp.py --brightness 50 images_input\mandrilc.bmp images_output\B1_out.bmp
  images_input\mandrilc.bmp -> path to image that will be changed

  images_output\B1_out.bmp -> path and name of newly generated image

  "50" -> amount added to each pixel value (if larger than 255 will be caped at 255)

  Working: The brightness function is iterating through the list of all pixels (of image) adding inserted number to each pixel value (or values in RGB case). To prevent exceeding 255 or dropping below 0, the MAX and MIN combination is used.

  Time efficiency: O(n) -> n number of pixels, the operation is single passing each image

  Memory efficiency O(n) -> n number of pixels, the program isn't editing existing image, but creating copy of them (with change)

Image processing	Task No. 1

Picture before:

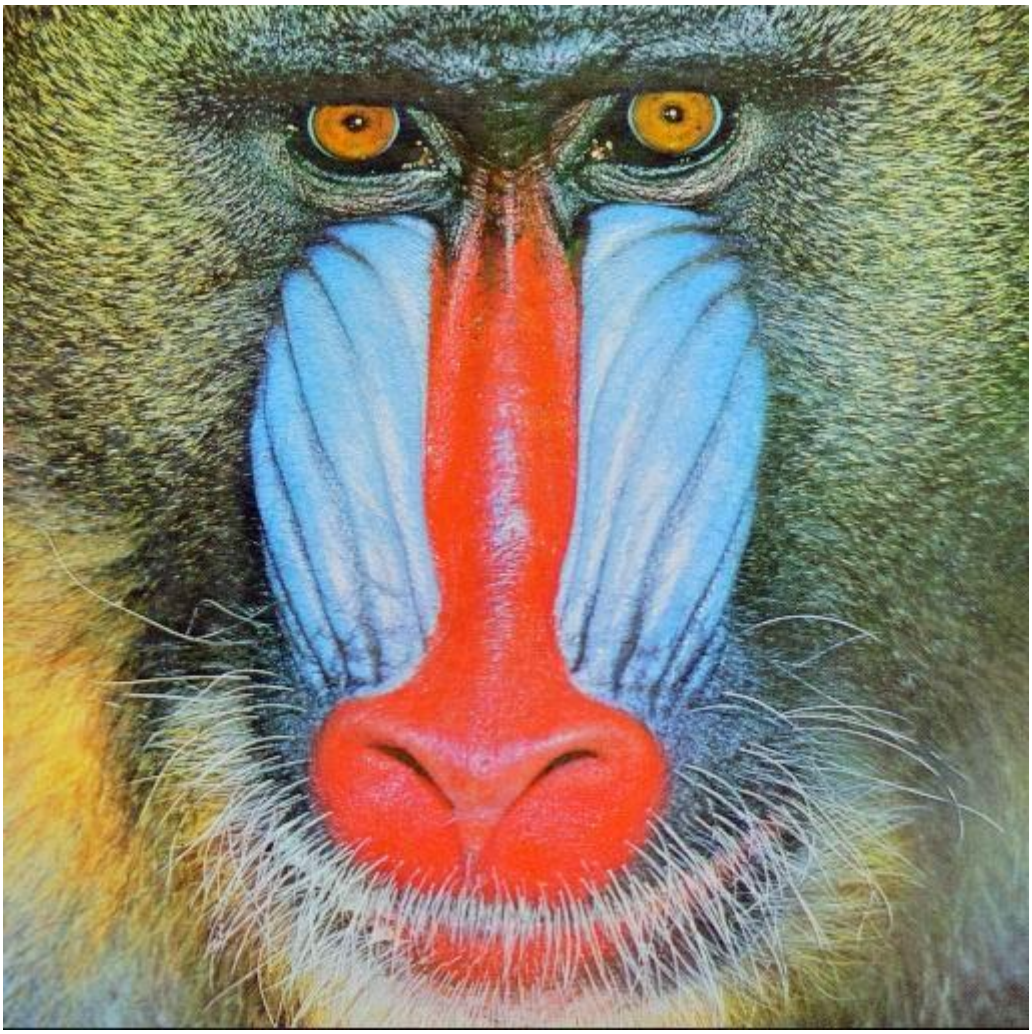
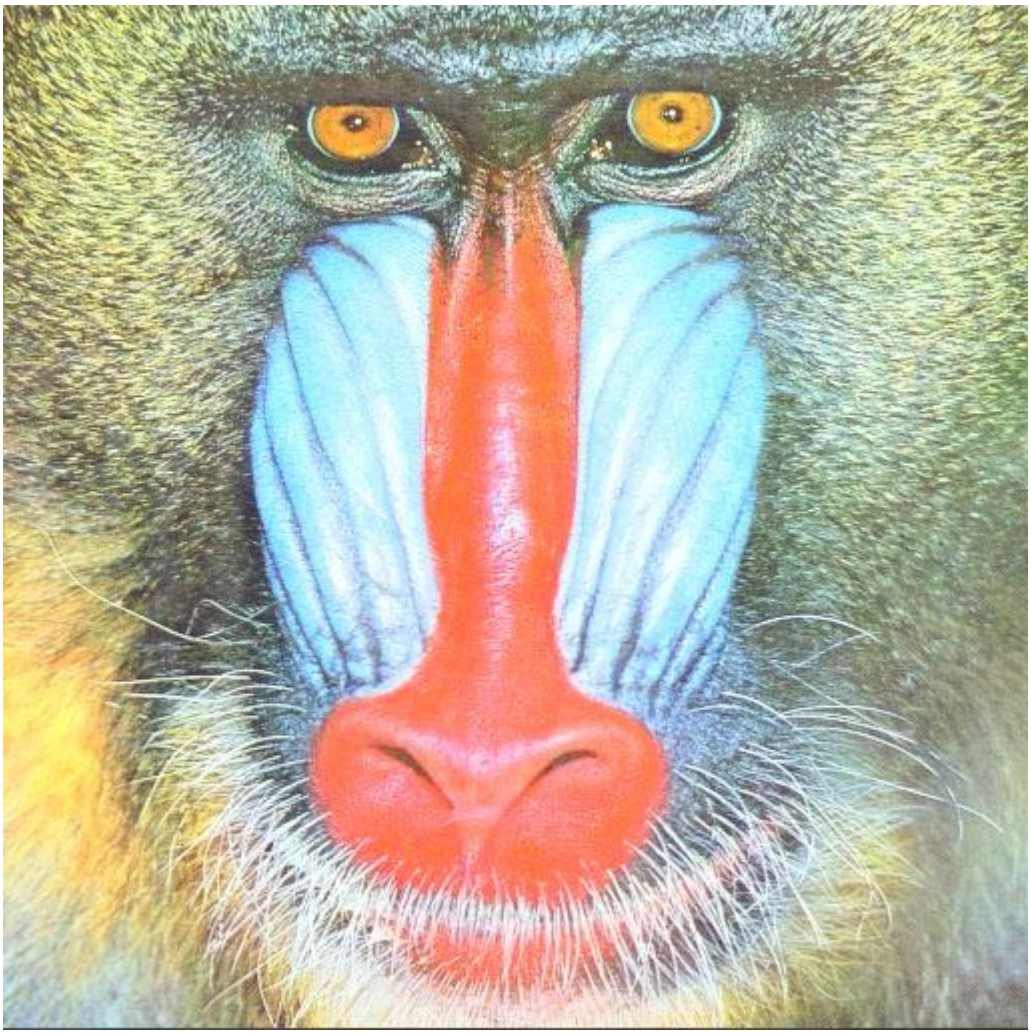


Image processing	Task No. 1

Picture after: 

- Description (B2)

  Command input: python myWonderfulImProcApp.py --contrast 1.5 images_input\mandrilc.bmp images_output\B2_out.bmp images_input\mandrilc.bmp -> path to image that will be changed

  images_output\B2_out.bmp -> path and name of newly generated image

  "1.5" -> factor used in calculating final value

  Working: The contrast function is iterating through the list of all pixels (of image) changing each pixel value firstly by centering with 128, later by putting factor and bringing 128 back. So the final function would look like: (value – 128)*factor + 128

  Time efficiency: O(n) -> n number of pixels, the operation is single passing each image

  Memory efficiency O(n) -> n number of pixels, the program isn't editing existing image, but creating copy of them (with change)

Image processing	Task No. 1

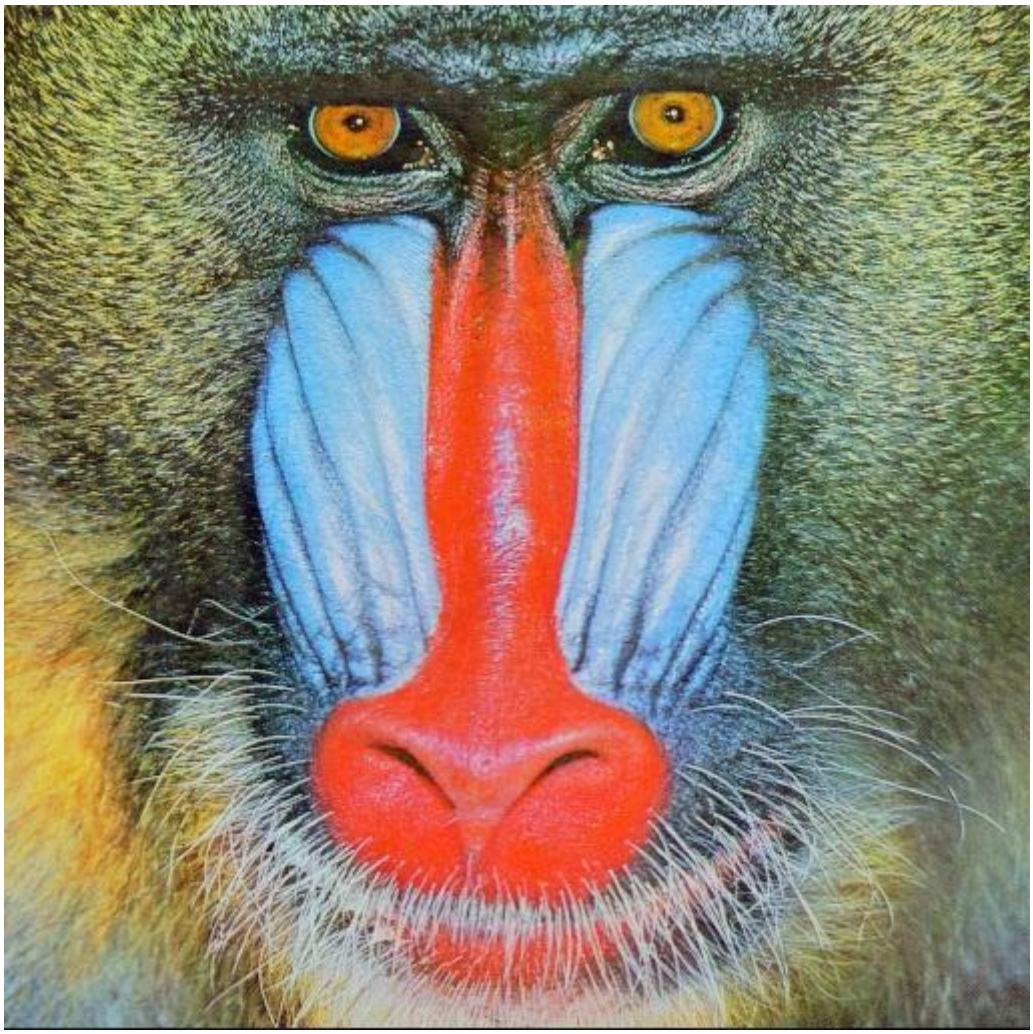
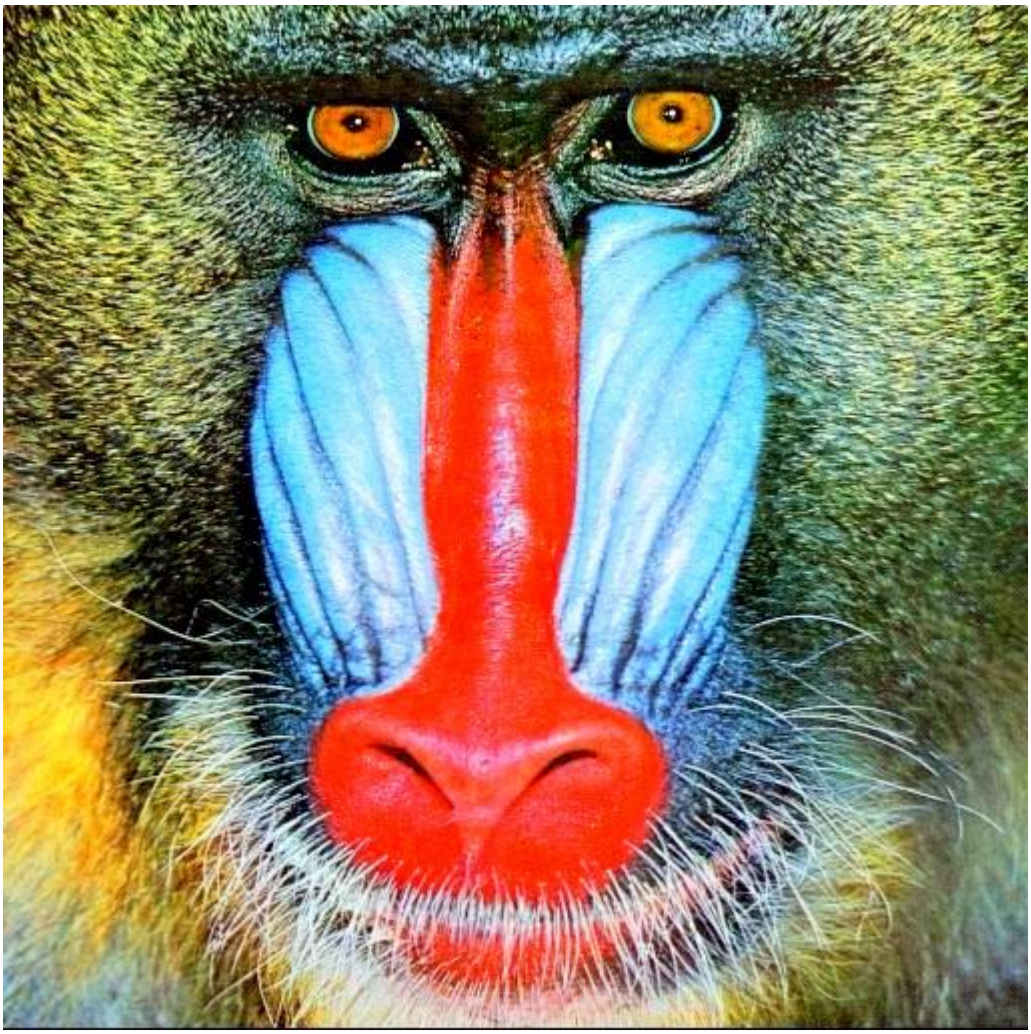
Picture before: 

Image processing	Task No. 1

Picture after: 

- Description (B3)

  Command input: python myWonderfulImProcApp.py --negative images_input\mandrilc.bmp images_output\B3_out.bmp
  images_input\mandrilc.bmp -> path to image that will be changed

  images_output\B3_out.bmp -> path and name of newly generated image

  Working: Negative functon is putting each pixel to negation. It is done by going through each pixel by iteration and changing it value by: 255-value

  Time efficiency: O(n) -> n number of pixels, the operation is single passing each image

  Memory efficiency O(n) -> n number of pixels, the program isn’t editing existing image, but creating copy of them (with change)

Image processing	Task No. 1

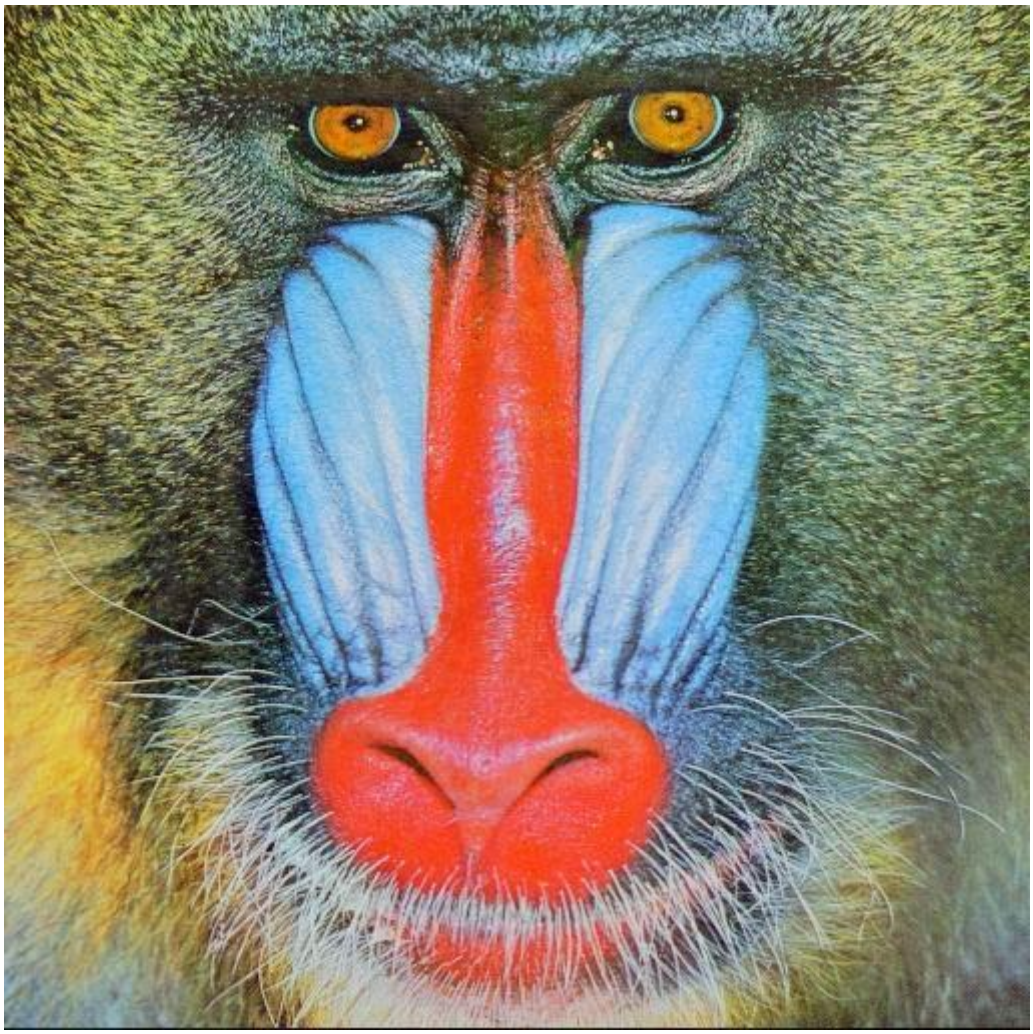
Picture before: 

Image processing	Task No. 1

Picture after: 

- Description (G1)

  Command input: python myWonderfulImProcApp.py --hflip images_input\mandrilc.bmp images_output\G1_out.bmp
  images_input\mandrilc.bmp -> path to image that will be changed

  images_output\G1_out.bmp -> path and name of newly generated image

  Working: Horizontal flip is going firstly row by row, calculating the index of last pixel of the row. Later it is saving pixels of each row backwards to new pixel list.

  Time efficiency: O(n) -> n number of pixels, the operation is single passing each image

  Memory efficiency O(n) -> n number of pixels, the program isn't editing existing image, but creating copy of them (with change)

Image processing	Task No. 1

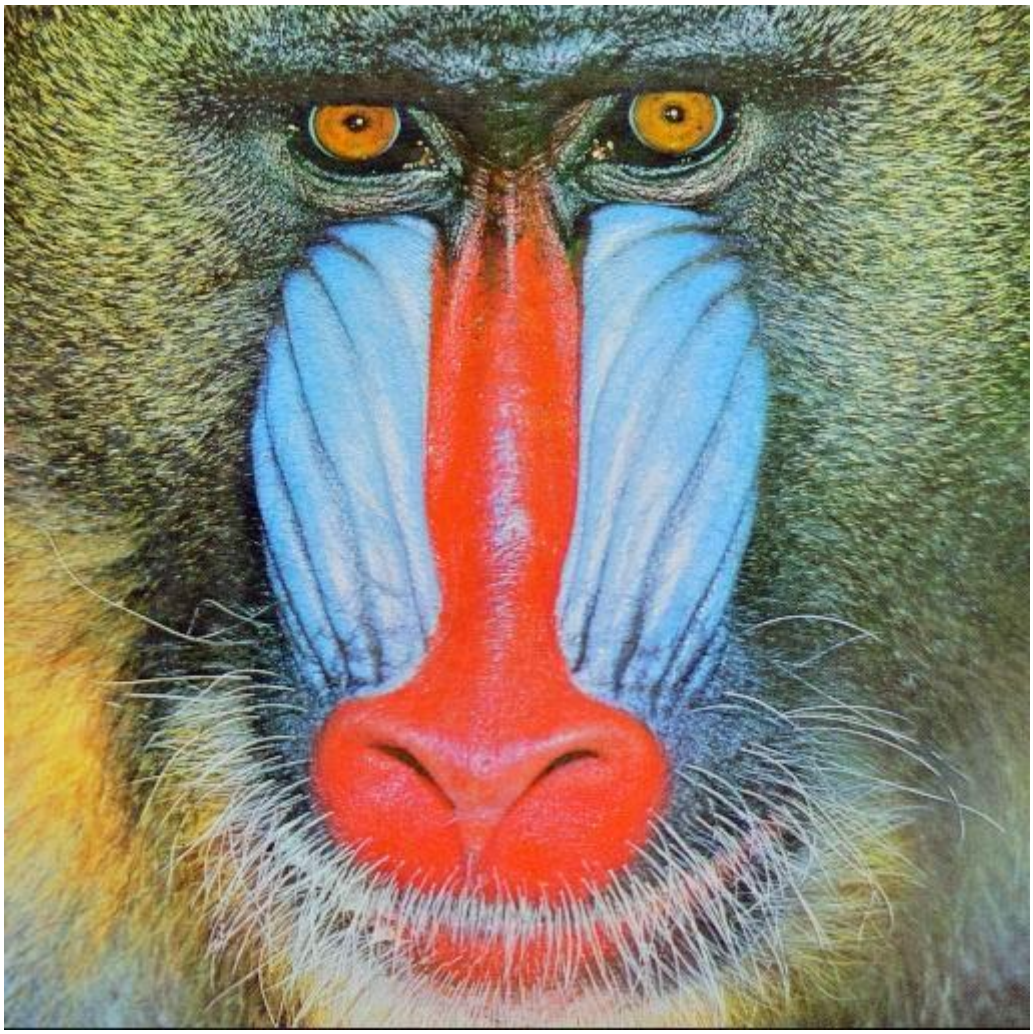
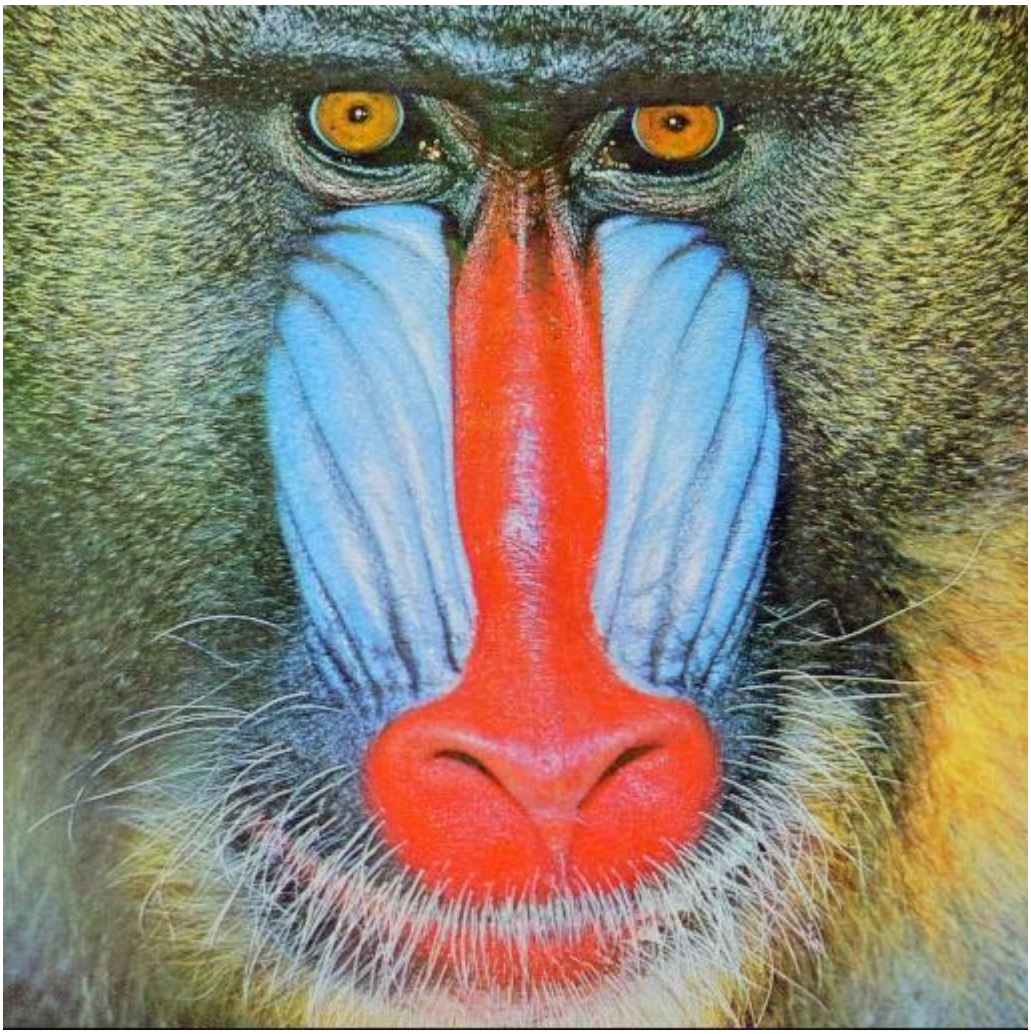
Picture before: 

Image processing	Task No. 1

Picture after: 

- Description (G2)

  Command input: python myWonderfulImProcApp.py --vflip images_input\mandrilc.bmp images_output\G2_out.bmp
  images_input\mandrilc.bmp -> path to image that will be changed

  images_output\G2_out.bmp -> path and name of newly generated image

  Working: Vertical flip is going firstly through the rows, calculating the starting pixel of all rows. Later it is saving pixels of each row in place of corresponding row (from bottom).

  Time efficiency: O(n) -> n number of pixels, the operation is single passing each image

  Memory efficiency O(n) -> n number of pixels, the program isn't editing existing image, but creating copy of them (with change)

Image processing	Task No. 1

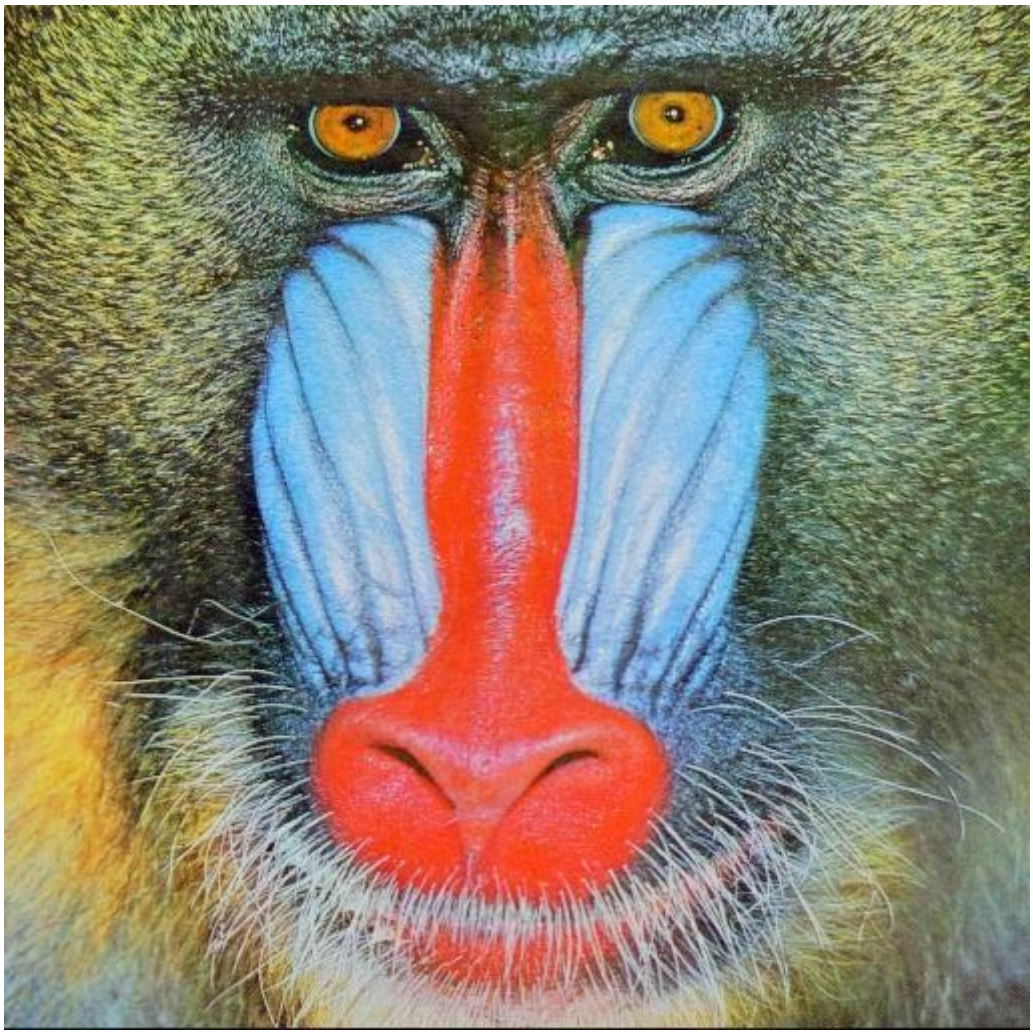
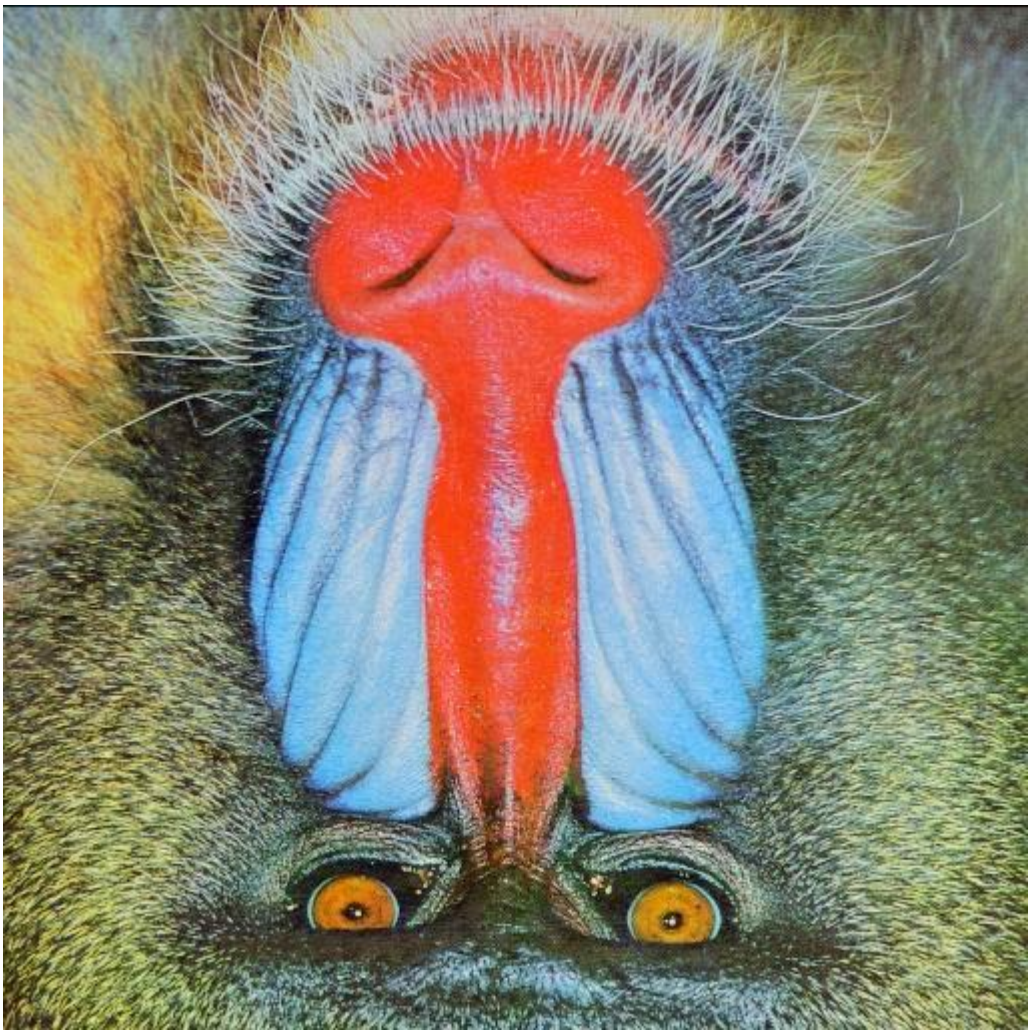
Picture before: 

Image processing	Task No. 1

Picture after: 

- Description (G3)

  Command input: python myWonderfulImProcApp.py --dflip images_input\mandrilc.bmp images_output\G3_out.bmp
  images_input\mandrilc.bmp -> path to image that will be changed

  images_output\G3_out.bmp -> path and name of newly generated image

  Working: Diagonal flip is going firstly through all rows, and then all columns applying horizontal flip on all rows and later vertical flip on all columns.

  Time efficiency: O(n) -> n number of pixels, the operation is single passing each image

  Memory efficiency O(n) -> n number of pixels, the program isn't editing existing image, but creating copy of them (with change)

Image processing	Task No. 1

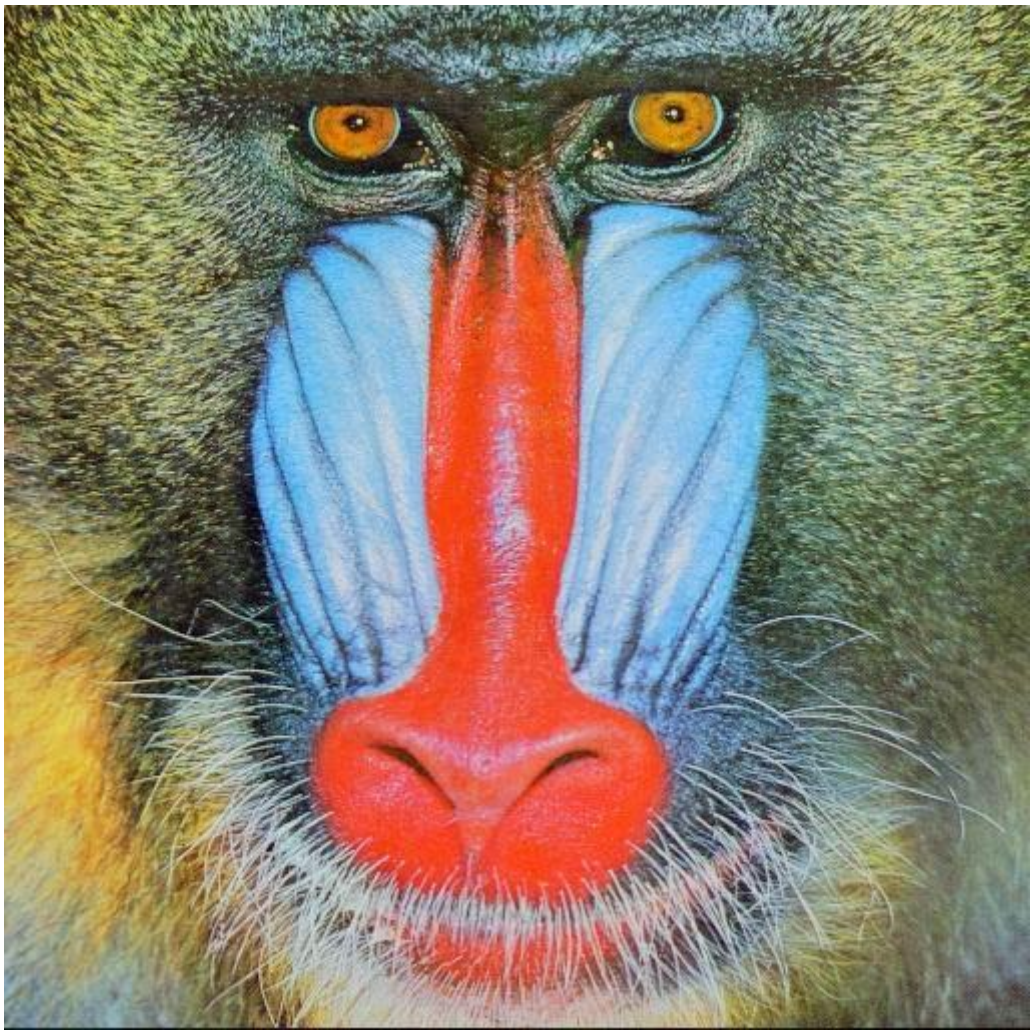
Picture before: 

Image processing	**Task No. 1**

Picture after:



- Description (G4)

  Command input: python myWonderfulImProcApp.py --shrink 2 images_input\mandrilc.bmp images_output\G3_out.bmp images_input\mandrilc.bmp -> path to image that will be changed

  images_output\G3_out.bmp -> path and name of newly generated image

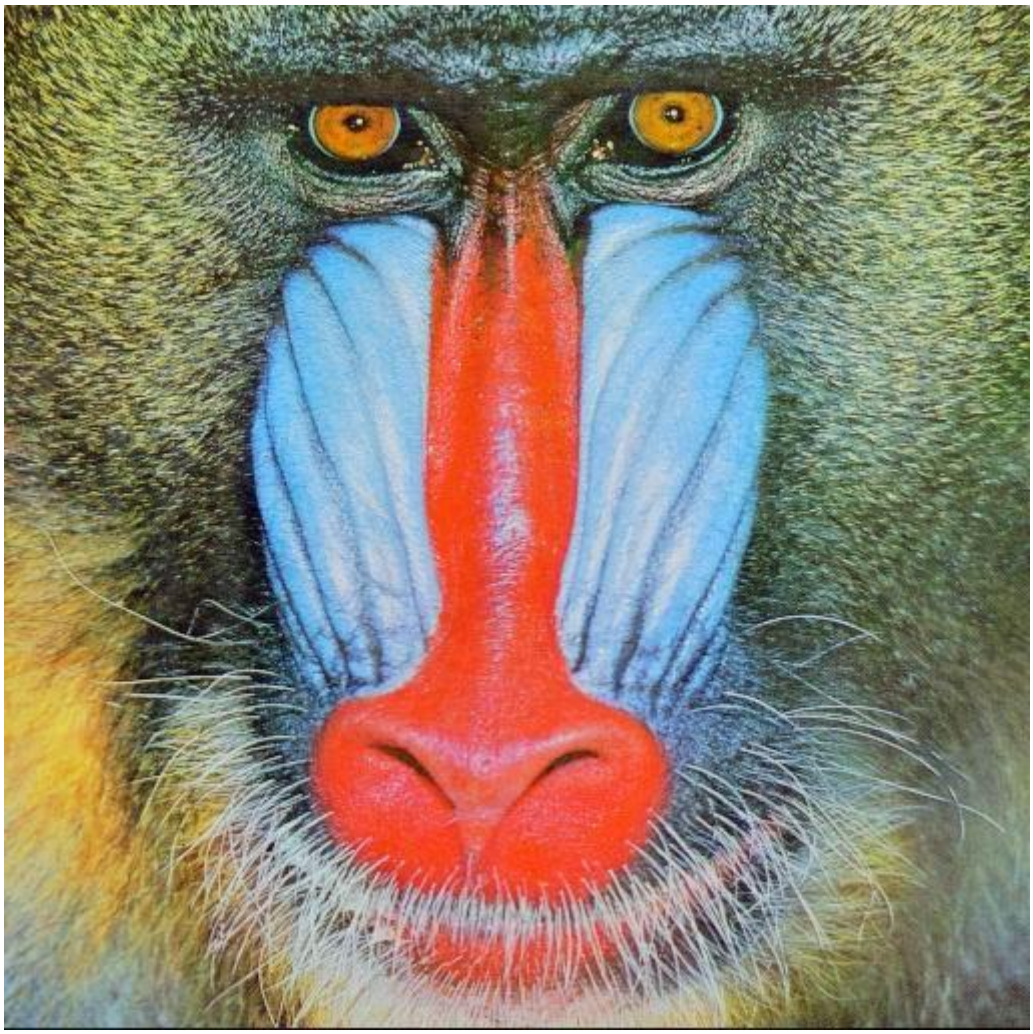
  "2" -> integer factor (shrinking image by dividing via this number)

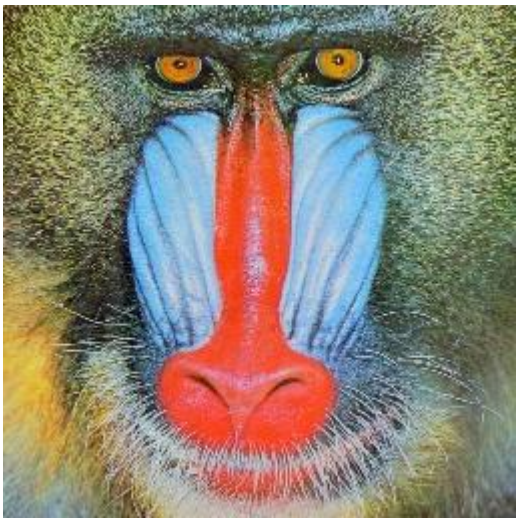
  Working: Shrink is firstly calculating new dimensions by dividing with shrink number. Later with saving to new image the program is skipping pixels, depended on factor (used with formula: (y*factor*width) +(x*factor)

  Time efficiency: O(n) -> n number of pixels of new image (output image after transformation), the operation is single passing each image

  Memory efficiency O(n) -> n number of pixels new image (output image after transformation), the program isn't editing existing image, but creating copy of them (with change)

Image processing	Task No. 1

Picture before: 

Picture after: 

- Description (G5)

  Command input: python myWonderfulImProcApp.py –-shrink 2 images_input\mandrilc.bmp images_output\G3_out.bmp images_input\mandrilc.bmp -> path to image that will be changed

  images_output\G3_out.bmp -> path and name of newly generated image

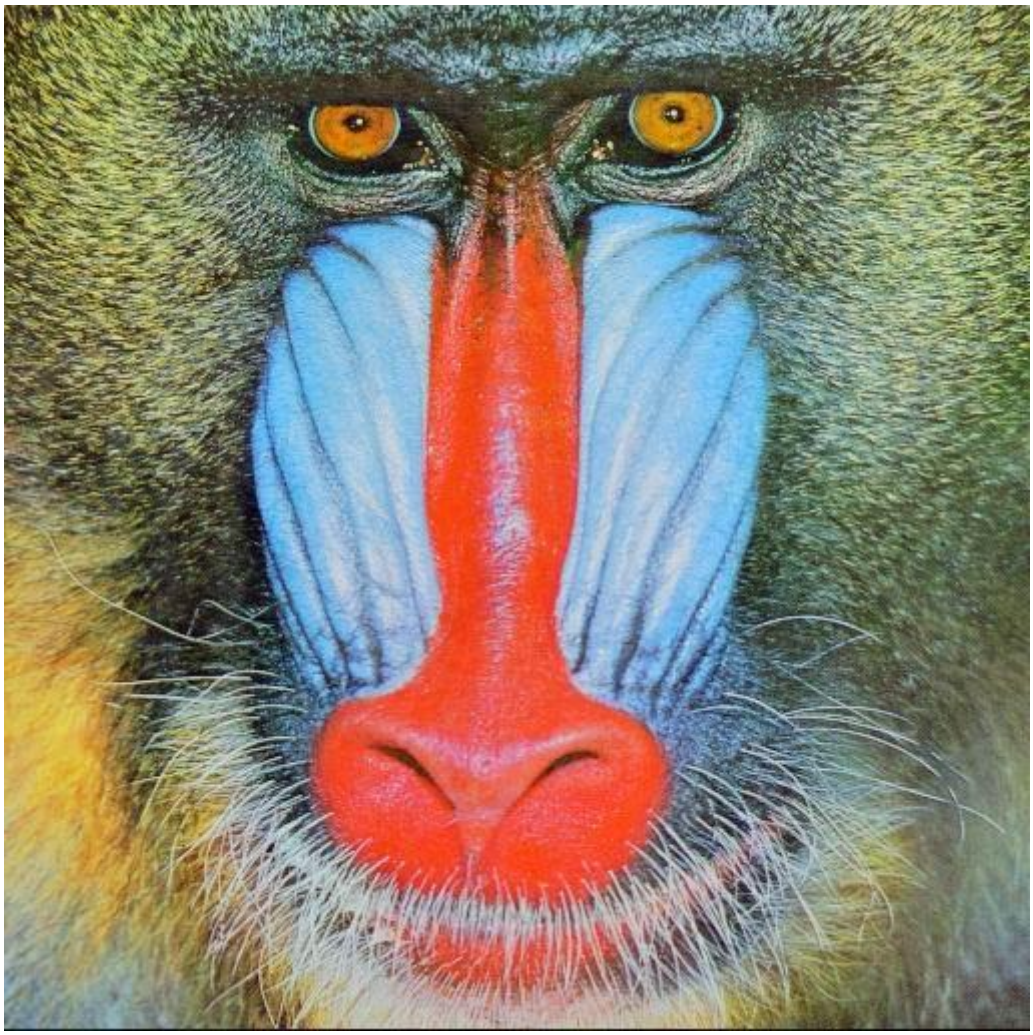
  "2" -> integer factor (enlarging image by multiplying via this number)

  Working: Enlarging is firstly calculating new dimensions by multiplying with enlarging number. Later with saving to new image

the program is filling newly created pixels with values of pixel in center (in case of 2 the one pixel will generate group of pixels of size 2x2)

ime efficiency: O(n) -> n number of pixels of new image (output image after transformation), the operation is single passing each image

Memory efficiency O(n) -> n number of pixels new image (output image after transformation), the program isn't editing existing image, but creating copy of them (with change)

Picture before: 

Picture after:

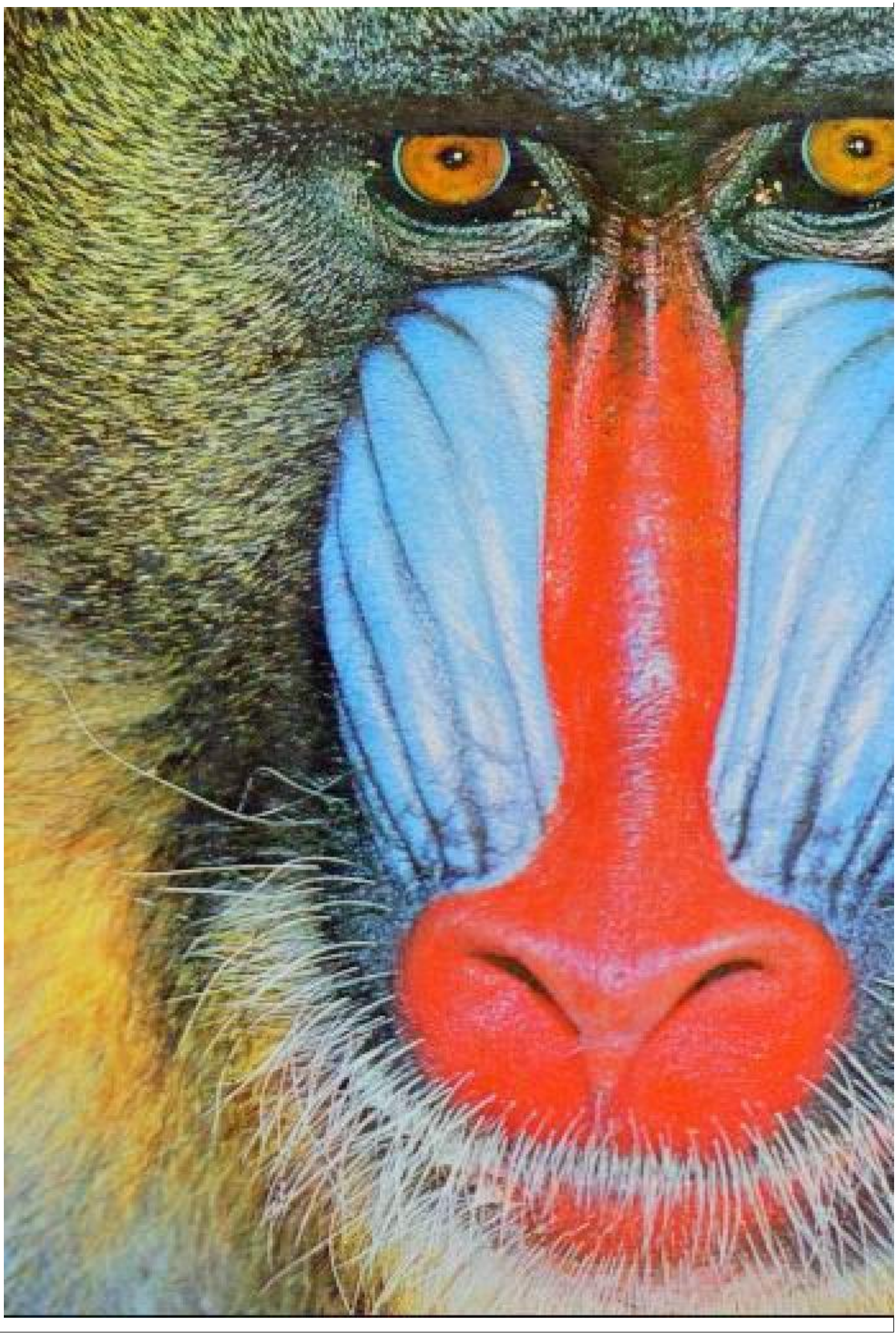


Image processing	Task No. 1

Here, a short description of implementation of the aforementioned basic operations should be placed. Attention should be paid (if possible) to the efficiency of this implementation w. r. t. both the time and the memory usage (these aspects should be described).

Description of implementation of noise reduction methods

- The first method:

  Min and Max filter (--min 3/--max 3), Where 3 is (usually) number of size of small squares we are dividing image into. After that the program iterate through every pixel in image with neighborhood (the divided squares), for out-of-range image boundaries, they are being clamped into nearest edge pixel. The values of pixels are chosen based on function in task description. The script enforces that the window size must be an odd integer greater than or equal to 3

  Time efficiency: O(n*x^2) -> x is the chosen window size, and n is number of pixels (it is cause each pixel is scanning whole window size (square 3x3 = 9 pixels)

  Memory efficiency O(n) -> n number of pixels new image (output image after transformation), the program isn't editing existing image, but creating copy of them (with change) + O(x^2) -> x chosen window size (temporarily we are coping squares as well)

- The second method

  Adaptive median filter (--adaptive 7). Where 7 is max window number. The filter is dynamically adjusting its size (starting from 3x3 going to chosen number). It extracts window and calculates min max and median. If median is impulse it is increasing window size until median isn't noise then return it, if median isn't noise it replace noise with it.

  Time efficiency: O(n*x^2*Log(x^2)) -> x is the chosen window size, and n is number of pixels (it is cause each pixel is scanning whole window size (square 7x7 = 49 pixels). And multiplying by logarithm of chosen max window size (the most pessimistic scenario)

  Memory efficiency O(n) -> n number of pixels new image (output image after transformation), the program isn't editing existing image, but creating copy of them (with change) + O(x^2) -> x chosen max window size (temporarily we are coping squares as well, and counting most negative scenario)

A short description of implementation of the assigned (in the variant N) noise reduction methods should be placed in this section. Attention should be paid (if possible) to the efficiency of this implementation w. r. t. both the time and the memory usage (these aspects should be described).

Analysis of parameters of the noise reduction methods

Results and conclusions

For Normal 2

Image processing	Task No. 1

Before: 

Image processing	**Task No. 1**

after --min 3: 

Image processing	Task No. 1

after --min 7: 

Image processing	**Task No. 1**

after –-max 3:



Image processing	Task No. 1

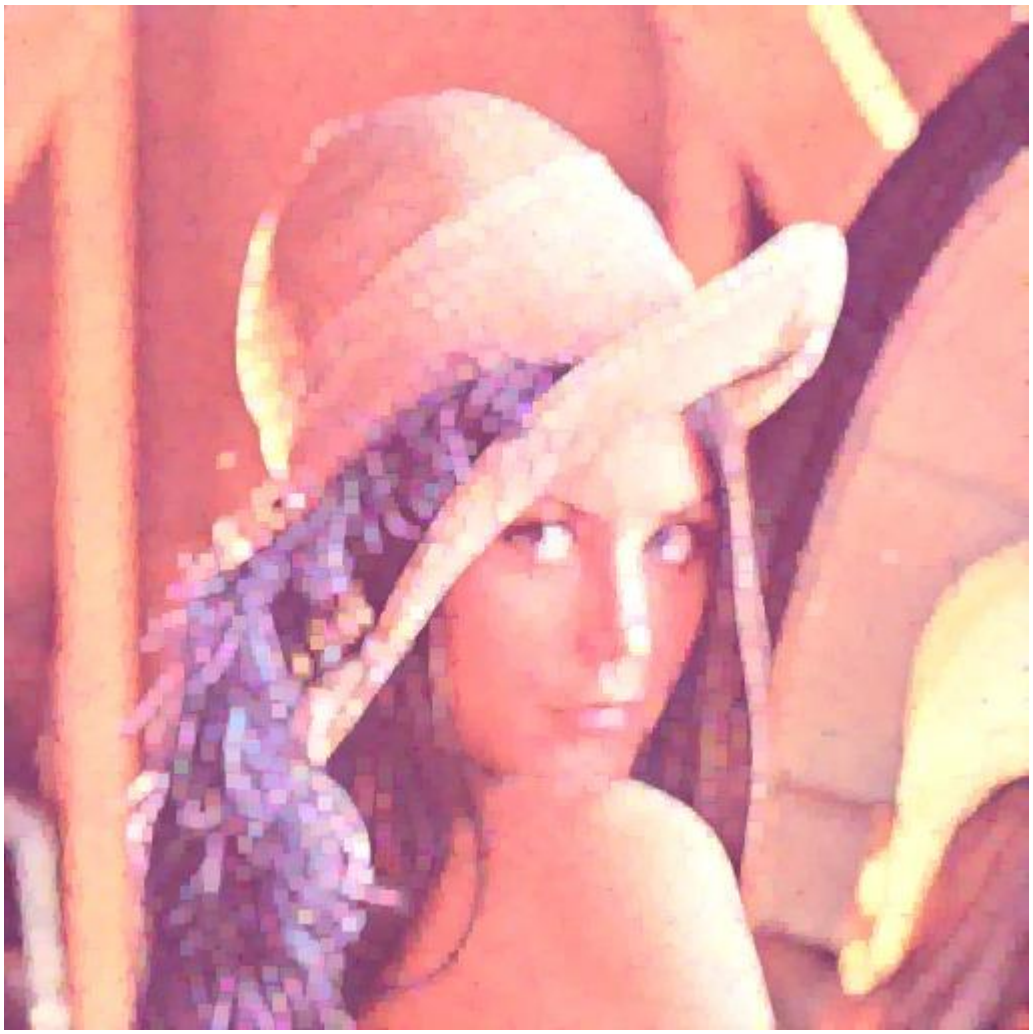
after --max 7: 

Image processing	Task No. 1

after –-adaptive 7: 

For Impulse 2

Image processing	Task No. 1

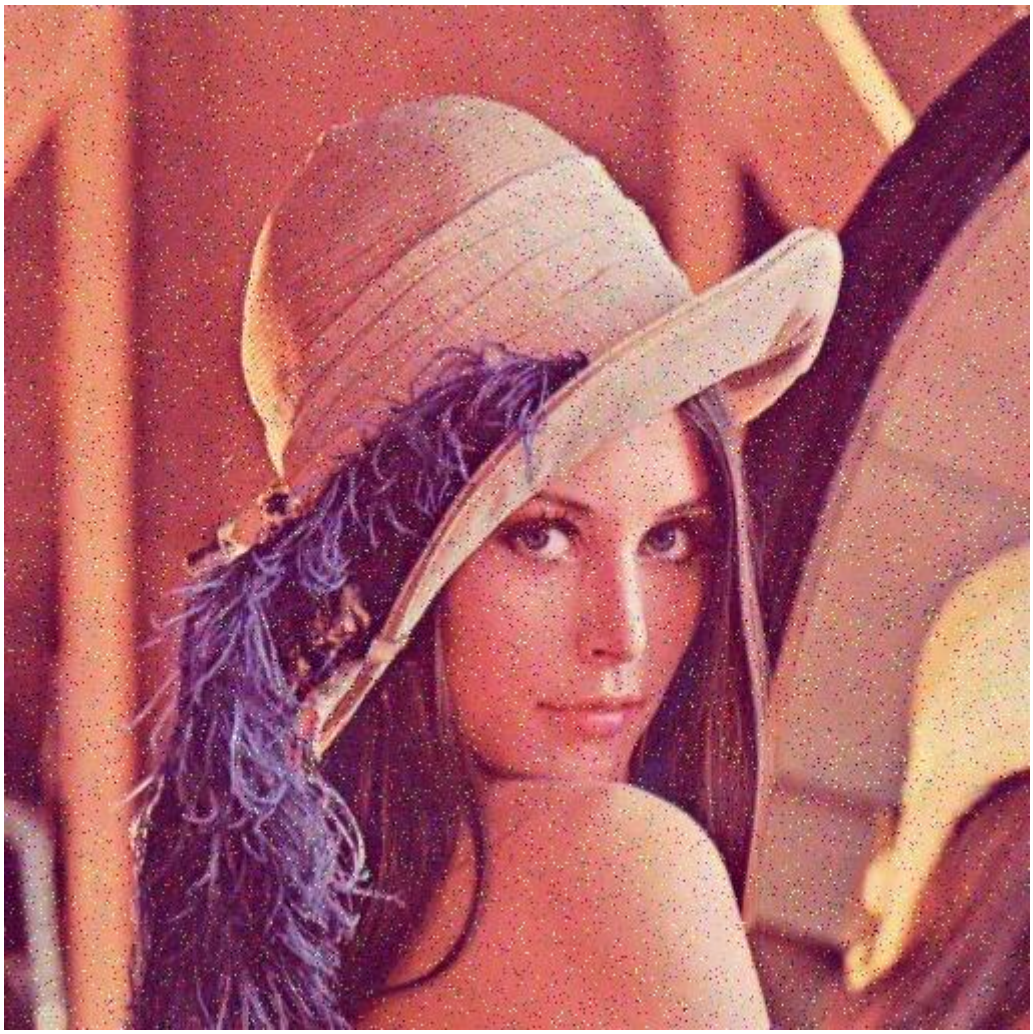
before: 

Image processing	**Task No. 1**

after --min 3:



Image processing	Task No. 1

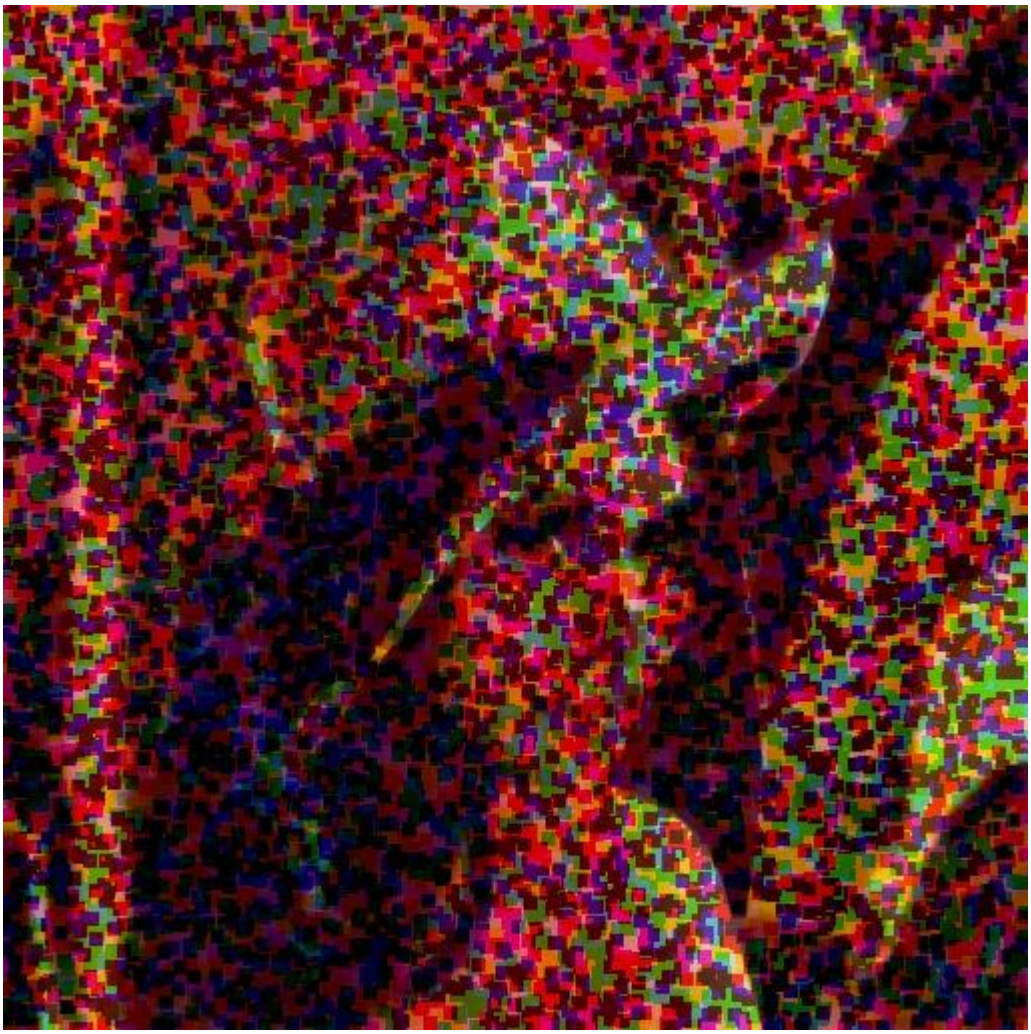
after --min 7: 

Image processing	**Task No. 1**

after –-max 3:



Image processing	Task No. 1

after --max 7:

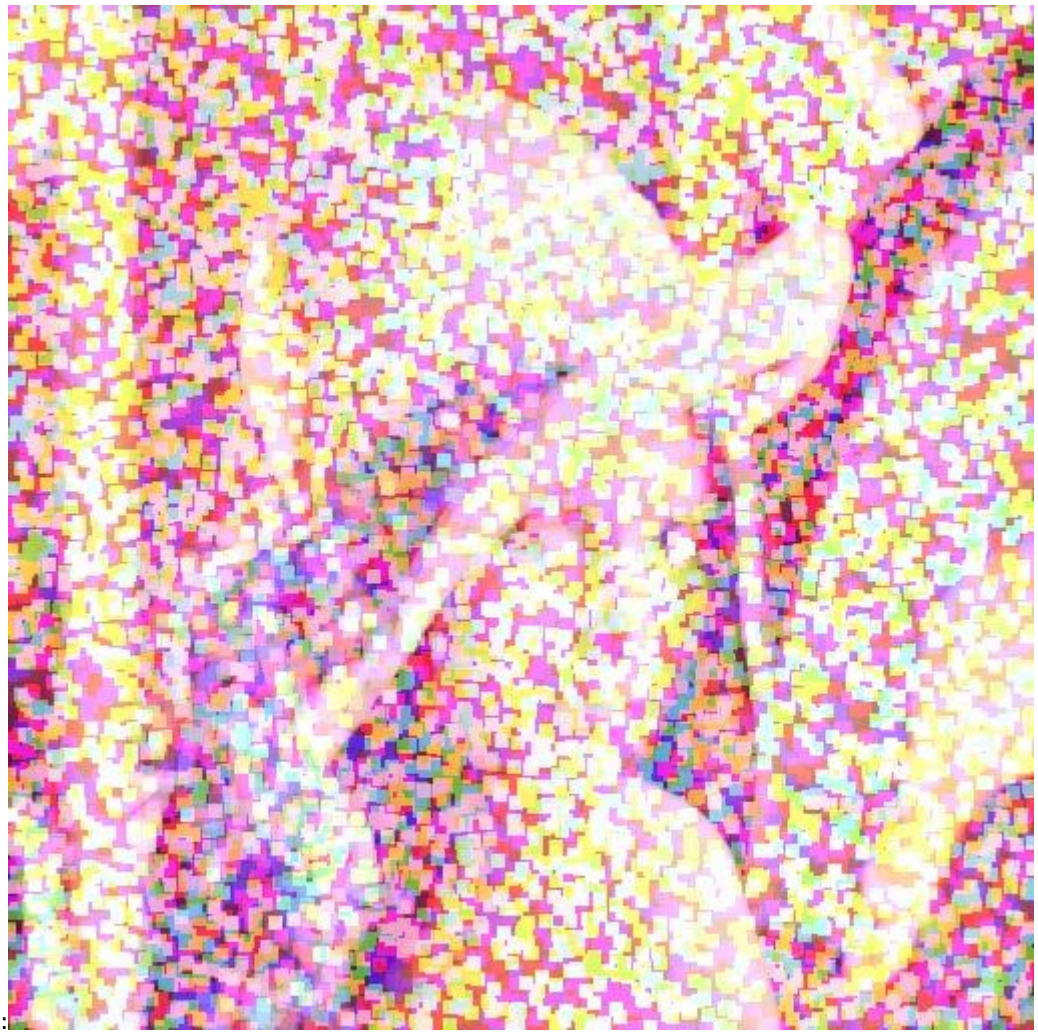


Image processing	**Task No. 1**

after –-adaptive 7: 

For Uniform 2

Image processing	Task No. 1

Before:



Image processing	Task No. 1

after --min 3: 

Image processing	**Task No. 1**

after --min 7: 

Image processing	**Task No. 1**

after –-max 3:



Image processing	Task No. 1

after –-max 7: 

Image processing	Task No. 1

after –-adaptive 7:



The results of the experiments related to changes in values of the parameters (if there are any) of the assigned noise reduction methods should be placed here. In this section the conclusions drawn from the experiments should be also placed.

Analysis of the noise reduction methods w. r. t. the possible applications for various types of noise

Results and conclusions

Normal 2 result:

Filter	PSNR	SNR	MD	PMSE	MSE
--Adaptive 7	23.166587661398	18.102258607292	114	0.0048232662245	313.63288625081
--Min 3	15.153302427420	10.088973373315	226	0.0305259899749	1984.9524981180
--Max 3	15.276395150246	10.212066096140	226	0.0296729336154	1929.4825083414

Impulse 2 result:

Filter	PSNR	SNR	MD	PMSE	MSE
--Adaptive 7	20.764203243498	15.703237922911	179	0.0083864792282	545.33081181844
--Min 3	13.808734618191	8.7477692976050	255	0.0416031809915	2705.2468439737
--Max 3	13.835015559859	8.7740502392727	255	0.0413521832783	2688.9257176717

Uniform 2 result:

Filter	PSNR	SNR	MD	PMSE	MSE
--Adaptive 7	26.455767190471	21.339857929900	113	0.0022616389811	147.06307474772
--Min 3	18.339566040201	13.223656779630	230	0.0146569429011	953.06771214803
--Max 3	18.484377415422	13.368468154850	230	0.0141762792149	921.81255594889

Image processing	**Task No. 1**
Objective (coefficients E) and subjective conclusions related to the quality of the results obtained by the assigned noise reduction methods for different types of noise should be placed here. The images provided on the web page should be applied for this purpose.	
Teacher's remarks This is a section for teacher's remarks for the laboratory group (please leave some free place).	